SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Scenario-Based : 20%

Total Marks: 50

Annexure A

Code of Conduct:

I R KALAI AMUDHAN (192511292) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

1. Secure Password Storage in Web Applications
A web application stores user passwords and needs to protect them against database breaches.

A. Explain why storing plaintext passwords is insecure.
B. Suggest a secure method for storing passwords and justify your choice.

2. Digital Signature for Document Authentication
A company wants to ensure that official documents sent electronically are authentic and unaltered.

A. Explain how digital signatures can be used in this scenario.
B. Justify how this method ensures both integrity and non-repudiation.

3. Preventing Replay Attacks in Online Transactions
An online payment system is vulnerable to replay attacks where attackers resend previously captured transactions.

A. Explain how replay attacks occur.
B. Suggest a cryptographic technique to prevent such attacks and justify your answer.

4. Secure Communication over Public Wi-Fi
A user accesses sensitive information using a public Wi-Fi network without encryption.

A. Identify the risks involved in this scenario.
B. Recommend a secure communication method and justify how it protects user data.

5. Data Integrity in Cloud Storage
An organization stores critical data in a cloud server and wants to ensure that the data is not altered.

A. Explain how hashing can be used to verify data integrity.
B. Suggest an additional mechanism to enhance trust and justify your answer.

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

ANSWERS:-

1. Secure Password Storage in Web Applications

A. Storing user passwords in plaintext means saving the exact characters entered by the user in the application database. This is a serious security weakness because anyone who gains unauthorized access to the database can immediately read the passwords without performing any additional cryptographic work. A database breach may occur due to software vulnerabilities, stolen administrator credentials, insecure backups, misconfigured cloud storage, or malicious insiders. In each case, plaintext passwords make the consequences of the breach much more severe.

The risk is not limited to the affected application. Many people reuse the same password on several websites. If an attacker obtains a password from one breached service, the attacker may try it on email, social media, banking, shopping, and college accounts. This is known as credential stuffing when stolen username-password combinations are tested against other services. A plaintext database therefore creates a risk of account takeover and identity theft beyond the original website.

Encryption alone is not the best primary solution for password storage if the application must later verify a login. Encrypted passwords can be decrypted if the decryption key is stolen or misused. Passwords generally do not need to be recovered in their original form; the application only needs to check whether the password entered during login matches the password originally registered. For that reason, the system should not store plaintext passwords and should not log passwords in application logs or expose them in error messages.

B. The recommended method is to use a dedicated password-hashing algorithm such as Argon2id, bcrypt, or scrypt, configured with an appropriate work factor. Password hashing is intentionally designed to make each password guess computationally expensive. A secure system generates a unique random salt for each password, processes the password and salt through the selected password-hashing algorithm, and stores the resulting hash together with the salt and algorithm parameters. The original password is discarded and cannot feasibly be recovered from the hash.

When the user logs in, the application retrieves the stored salt and parameters and runs the password entered by the user through the same password-hashing process. The system verifies the result against the stored hash using the algorithm's recommended verification procedure. If the result matches, the password is accepted; otherwise, access is denied. The server should never need to decrypt or display the user's original password.

A unique salt is important because two users who choose the same password should not have identical stored password hashes. Salts also make precomputed rainbow-table attacks much less effective. The work factor or cost setting slows down password guessing, increasing the attacker's time and computing cost if the database is stolen. A fast general-purpose hash such as plain SHA-256 should not be used alone for password storage, because attackers can test huge numbers of guesses quickly with specialized hardware.

Additional safeguards include enforcing reasonable password length, blocking commonly used or compromised passwords, using multi-factor authentication, limiting repeated login attempts, monitoring suspicious activity, protecting database backups, and restricting database access. Passwords should be transmitted only over HTTPS and should never be written to logs. Thus, salted and appropriately configured password hashing is the most suitable core storage method because it reduces the impact of a database breach while allowing the application to verify users securely.

2. Digital Signature for Document Authentication

A. A digital signature is a cryptographic mechanism used to check the authenticity and integrity of an electronic document. In this scenario, a company can use digital signatures for contracts, invoices, official letters, purchase orders, reports, or other documents that must be shown to come from an authorized person and remain unchanged during transmission.

The process begins when the sender completes the document. A cryptographic hash function, such as SHA-256, is applied to the document to produce a fixed-length digest. The sender then uses a digital-signature algorithm, such as RSA-PSS or ECDSA, with the sender's private key to create a signature associated with that digest. The sender transmits the document and its signature to the recipient. The private key must remain confidential and should be protected using appropriate access controls or secure key-storage facilities.

When the recipient receives the document, the recipient obtains the sender's trusted public key and uses it to verify the signature. The recipient also calculates a fresh hash of the received document. The digital-signature verification process checks whether the signature is valid for the received content and the corresponding public key. If verification succeeds, the recipient has evidence that the document has not changed since it was signed and that the signature was generated using the private key corresponding to the public key used for verification. If the document is altered, verification should fail.

For a business deployment, the company should establish trust in the public key. This may be done using a valid digital certificate issued by a trusted Certificate Authority or another trusted key-distribution process. The recipient should check certificate validity, the identity represented by the certificate, and revocation status where appropriate. The company should also preserve the signed document and relevant audit information.

B. Digital signatures support integrity because the signature is bound to the document's hash. If an attacker changes even a small part of the document—for example, changing an invoice amount from ₹10,000 to ₹90,000—the hash calculated by the recipient will differ from the hash associated with the original signature. The altered document will therefore fail signature verification. This allows the recipient to detect unauthorized modification rather than relying only on the sender's claim that the document is unchanged.

Digital signatures support authentication because successful verification demonstrates that the signature was produced using the private key corresponding to the public key used in verification. When that public key is reliably linked to the sender's identity, the recipient can associate the signed document with that sender. However, this conclusion depends on the private key being protected and the public key or certificate being trusted. If a private key is stolen, an attacker may create signatures that appear valid until the compromise is discovered.

Digital signatures also support non-repudiation. Non-repudiation means providing evidence that makes it difficult for a signer to credibly deny having signed a document. A valid signature, a trusted certificate, secure private-key management, timestamps where suitable, and reliable audit records can collectively support this evidence. Non-repudiation is not an absolute guarantee in every dispute; the organization must also consider who controlled the private key, whether it was compromised, and whether signing procedures were properly followed.

A digital signature does not hide the document's contents. Anyone who can access the transmitted file may still be able to read it unless encryption is used separately. Therefore, the company should use a digital signature when it needs authenticity and integrity, and apply encryption as well if confidentiality is required. This combination provides a stronger solution for official electronic documents.

3. Preventing Replay Attacks in Online Transactions

A. A replay attack occurs when an attacker captures a legitimate message or transaction and transmits the same message again in an attempt to make the receiver repeat an action. The attacker may not need to understand or decrypt the message. If the captured transaction is still accepted as valid, the attacker can reuse it. Replay attacks are especially concerning in payment systems, banking applications, authentication protocols, and systems that accept commands over a network.

For example, suppose a customer sends a valid request to transfer ₹2,000 from one account to another. An attacker monitoring an inadequately protected network records the transaction request. If the payment server accepts the same request repeatedly without checking whether it has already been processed, the attacker may resend it and cause a duplicate transfer. The same problem could occur with a repeated online purchase, a repeated account action, or a reused authentication message.

A replay attack differs from simply modifying a message. The attacker may send the original message exactly as it was captured. Consequently, a valid digital signature or message authentication code (MAC) alone may not prevent replay: the original message can still have a valid signature or MAC when it is resent. The main weakness is that the system has not established that the request is fresh and has not already been used.

B. A suitable solution is to combine cryptographic authentication with freshness checks, such as a unique nonce or transaction identifier, and server-side duplicate detection. A nonce is a value intended to be unique for a particular protocol operation. A transaction ID uniquely identifies a payment request. The system should include the nonce or transaction ID within the authenticated data, so an attacker cannot change it without invalidating the signature or MAC.

A secure transaction can include the payer's account, recipient's account, amount, currency, timestamp, and unique transaction ID. The client signs the complete transaction data with a private key, or authenticates it with a suitable MAC when the protocol uses a shared secret. The server verifies the signature or MAC, checks that the timestamp is within the permitted validity period, and checks whether the transaction ID has already been processed.

Timestamps can help reject old messages, but they require reliable clock handling and a defined time window. A timestamp alone may not stop an attacker from replaying the same request within that window, so it should be combined with a nonce, transaction ID, or replay cache. The server must record accepted identifiers reliably and make the check-and-process operation atomic; otherwise, two simultaneous copies of a request might both be processed.

TLS is also important because it protects data in transit and helps prevent attackers from reading or modifying network traffic. However, TLS should not be treated as a replacement for application-level replay protection in payment workflows. A well-designed system combines encrypted transport, authenticated transaction details, freshness validation, duplicate detection, authorization checks, and audit logging. This approach is justified because a replayed message may remain cryptographically authentic, but it will not be accepted as a new transaction once the server recognizes its reused identifier.

4. Secure Communication over Public Wi-Fi

A. Public Wi-Fi networks are often available in airports, cafés, hotels, libraries, shopping centres, and campuses. When a user accesses sensitive information over a connection that does not encrypt the communication, the user's data may be exposed to other people connected to the same network or to an attacker who can observe the traffic. The main risks include eavesdropping, credential theft, man-in-the-middle attacks, fake hotspots, session theft, and data manipulation.

First, eavesdropping occurs when an attacker captures network traffic and reads information that is transmitted without encryption. This may expose usernames, passwords, messages, personal details, or other confidential information. Second, an attacker may set up a fake Wi-Fi hotspot with a name similar to the genuine network. A user who connects to the fake hotspot may unknowingly send traffic through the attacker-controlled device.

Third, a man-in-the-middle attack occurs when an attacker positions themselves between the user and the intended service, attempting to observe or modify communication. The attacker might redirect the user to a fraudulent website, alter unprotected content, or attempt to impersonate a legitimate server. Fourth, session information such as cookies may be at risk if a service does not protect them properly, potentially allowing an attacker to misuse an authenticated session. These risks can lead to account takeover, privacy loss, financial fraud, and unauthorized changes to information.

The risk is particularly serious when the user accesses banking services, email, cloud storage, college portals, or business systems. Even if a website uses encryption, a user may still be deceived by a fake website or may ignore certificate warnings. Public Wi-Fi is not automatically unsafe in every case, but the user should not assume that the network operator or every connected device is trustworthy.

B. The primary recommendation is to use HTTPS protected by Transport Layer Security (TLS) for websites and applications. TLS encrypts the data exchanged between the user's device and the server, helping prevent nearby network users from reading the contents. It also provides integrity protection to detect unauthorized modification and uses server certificates to help the client verify that it is communicating with the intended server.

The user should confirm the website address, avoid clicking suspicious links, and never bypass browser warnings about invalid or untrusted certificates. The user should prefer official applications and websites, disable automatic connection to unknown Wi-Fi networks, and disconnect from hotspots with suspicious login pages. A reputable Virtual Private Network (VPN) can create an encrypted tunnel between the device and the VPN provider, reducing exposure to local-network monitoring. However, a VPN shifts trust to the VPN provider and does not guarantee that a fraudulent website is safe; HTTPS remains important.

Other recommended safeguards include enabling multi-factor authentication, keeping the operating system and browser updated, using a password manager, avoiding sensitive transactions on suspicious networks, and signing out of shared devices. Users should also avoid installing certificates or applications prompted by an unknown hotspot. Organizations can strengthen protection through secure application design, strong authentication, and properly configured TLS.

These measures are appropriate because encryption protects confidentiality, integrity checks help detect tampering, and certificate validation helps authenticate the server. Nevertheless, TLS cannot protect data if the user's device or the destination server is already compromised. Secure communication therefore requires both cryptographic protection and careful user behaviour.

5. Data Integrity in Cloud Storage

A. Data integrity means ensuring that information remains complete and unaltered except through authorized changes. An organization storing important files in a cloud server may need to detect accidental corruption, transmission errors, unauthorized modification, or malicious tampering. A cryptographic hash function provides a way to calculate a fixed-length digest that represents the contents of a file.

For example, when the organization uploads a report, it calculates the report's SHA-256 hash and stores that value in a protected verification record. Later, when the report is downloaded or checked, the organization calculates the hash of the retrieved file using the same algorithm. It compares the newly calculated hash with the previously recorded hash. If the values match, this provides strong evidence that the file contents are unchanged. If the values differ, the file may have been modified, corrupted, or replaced, and the organization should investigate before relying on it.

A cryptographic hash is designed so that it is computationally infeasible to find an input that matches a specified hash or to find two different inputs with the same hash in practical circumstances for a secure algorithm. Even a small change to the file, such as changing a number in a spreadsheet or a sentence in a report, is expected to produce a different digest. Hashing is therefore useful for checking downloaded backups, software packages, archived documents, and cloud-stored records.

However, a hash has limitations. A hash does not encrypt the file and therefore does not provide confidentiality. It also does not, by itself, identify who created or changed the file. Most importantly, if an attacker can modify both the file and the stored reference hash, the attacker may replace both values and make a simple comparison appear successful.

B. An additional mechanism is to digitally sign the hash using the organization's private key. The organization calculates the file's hash and signs the digest. The signature and file are stored, while the corresponding public key is made available through a trusted process. During verification, the organization checks the digital signature using the trusted public key and independently recalculates the file's hash. It then compares the recalculated hash with the digest covered by the valid signature. If an attacker changes the file, the new hash will not match the signed digest; if the attacker tries to replace the digest, the attacker cannot create a valid signature without the private key.

A keyed-hash message authentication code (HMAC) is another option when the organization and verifier securely share a secret key. An HMAC authenticates the data using a cryptographic hash and the secret key, making it more reliable than an unprotected hash when the key remains secret. Digital signatures are particularly useful when verification must be performed by parties that should not possess the signing private key, because they can verify using the public key.

The organization should also use strong access controls, multi-factor authentication for administrators, encryption for sensitive data, version history, protected audit logs, and immutable or write-once storage for critical records. Backups should be kept separately and restoration should be tested. Private signing keys and HMAC secrets must be protected, and key rotation and revocation procedures should be planned. These controls reduce the chance that an attacker can modify the file and its verification evidence at the same time.

Therefore, hashing is an efficient way to detect changes in file contents, while a digital signature or HMAC helps protect the authenticity of the integrity check. Combined with access control, secure key management, audit trails, and backups, these measures provide a stronger and more trustworthy cloud-storage solution.